

Cairo University

Faculty of Computers and Artificial Intelligence



CS251

Introduction to Software Engineering

MASROFY

Software Design Specifications

Version 1.0

20240741	Ahmed Sherif Ahmed	20240741@stud.fci-cu.edu.eg	01023417803
20240227	Sherif Ahmed Metwally	20240227@stud.fci-cu.edu.eg	01221406803
20240419	Karim Ashraf Ahmed	20240419@stud.fci-cu.edu.eg	01288227949
20240097	Eyad Hatem Fouad	20240097@stud.fci-cu.edu.eg	01279142909

April 2026



CS251: Phase1-
Project: Masroofy

Software Design Specification

Contents

Team	3
Document Purpose and Audience	3
System Models	4
I. Architecture Diagram	4
II. Class Diagram(s).....	5
III. Class Descriptions	7
IV. Sequence diagrams	12
Class – Sequence Usage Table	15
V. State Diagram	19
VI. Solid Principles.....	19
VII. Design Patterns	19
Tools	20
Ownership Report	20



CS251: Phase1-
Project: Masroofy

Software Design Specification

Team

20240097	Eyad Hatem Fouad	20240097@stud.fci-cu.edu.eg	01279142909
20240227	Sherif Ahmed Metwally	20240227@stud.fci-cu.edu.eg	01221406803
20240419	Karim Ashraf Ahmed	20240419@stud.fci-cu.edu.eg	01288227949
20240741	Ahmed Sherif Ahmed	20240741@stud.fci-cu.edu.eg	01023417803

Document Purpose and Audience

Purpose: This document specifies the software design for the Masroofy application, translating the requirements defined in the SRS into a concrete architectural and structural blueprint. It describes the system architecture, class diagram, sequence diagrams, state diagram, and the design patterns and SOLID principles applied, to guide the development team during implementation.

Audience: The primary audience includes the Course Instructor (Dr. M. El-Ramly), the TA, and the development team.

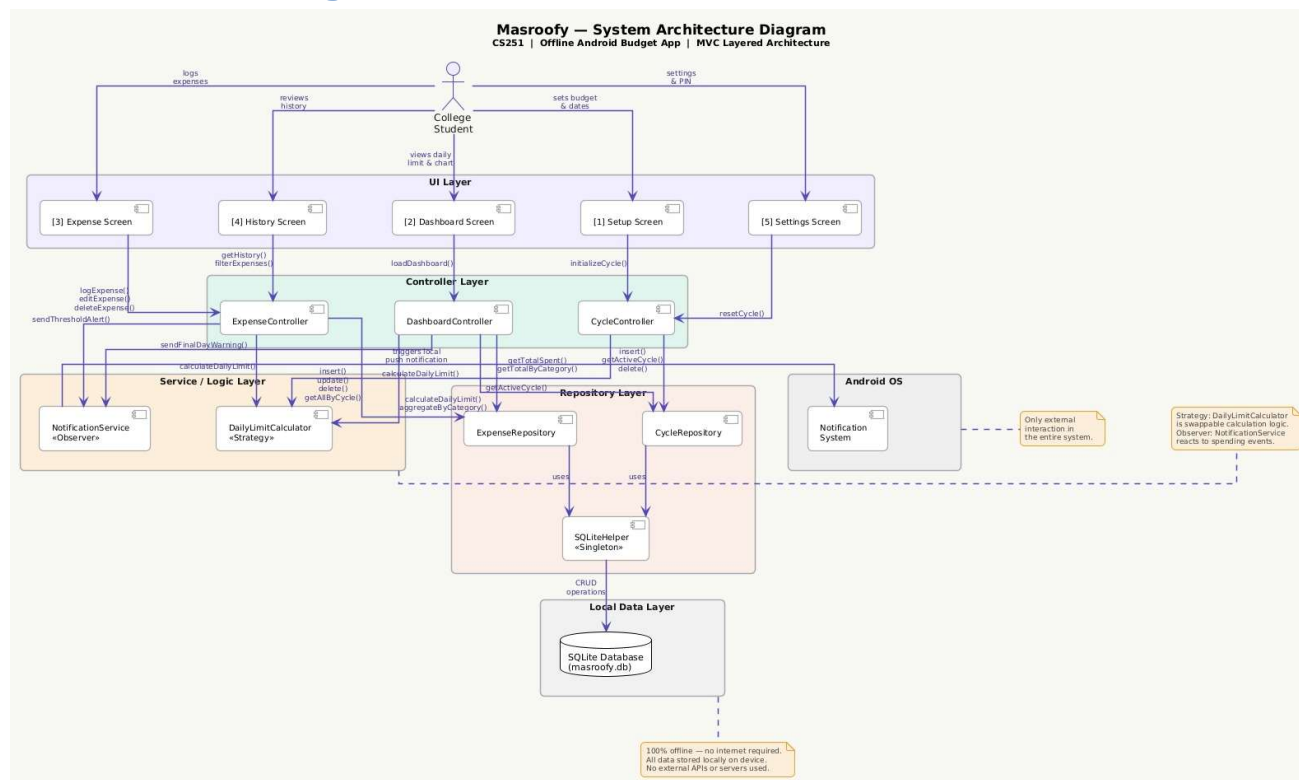


CS251: Phase1- Project: Masroofy

Software Design Specification

System Models

I. Architecture Diagram

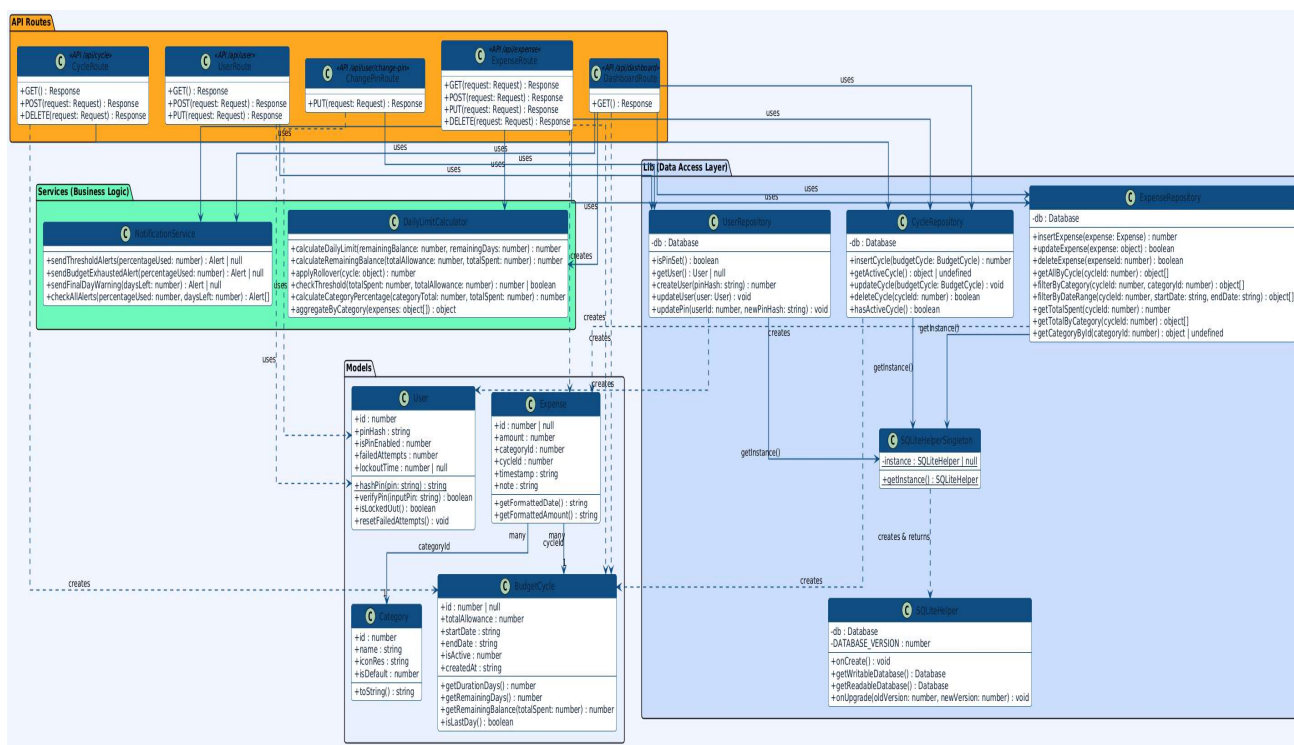


II. Class Diagram(s)



Software Design Specification

BONUS one: (More Specific For implementation with Next.js):





Software Design Specification

III. Class Descriptions

Model Package	Service Package	Repository Package	Controller Package
---------------	-----------------	--------------------	--------------------

Class Name	Responsibilities	Collaborators	Pattern / Notes
◆ Model Package			
BudgetCycle	• Stores total allowance, start date, end date, and active status • Calculates total duration in days (getDurationDays) • Calculates remaining days until cycle end (getRemainingDays) • Calculates remaining balance after all expenses (getRemainingBalance) • Determines whether today is the final day of the cycle (isLastDay)	• Expense (owns via composition) • CycleRepository • DailyLimitCalculator	Core domain object. Owns Expense via 1..* composition. Source: US#1, FR-1, FR-2
Expense	• Stores amount, category reference, cycle reference, and timestamp • Stores optional user note for context • Formats date for display in history screen (getFormattedDate) • Formats amount in EGP for display (getFormattedAmount)	• BudgetCycle (belongs to) • Category (belongs to) • ExpenseRepository	Belongs to one BudgetCycle and one Category. Source: US#2, US#7, FR-3
Category	• Stores category name (Food, Transport, Entertainment, Other) • Stores icon resource reference for UI display • Flags whether category is a default or user-created • Provides string representation for display (toString)	• Expense (referenced by many) • ExpenseRepository	Lookup / reference object. Cannot be deleted if expenses reference it. Source: US#2, US#9



Software Design Specification

Class Name	Responsibilities	Collaborators	Pattern / Notes
User	• Stores hashed PIN for local authentication • Tracks PIN enabled/disabled state • Counts consecutive failed login attempts • Records lockout timestamp to enforce 30-second lockout • Verifies entered PIN against stored hash (verifyPin) • Checks if account is currently in lockout period (isLockedOut) • Resets failed attempt counter after successful login (resetFailedAttempts)	• CycleController (authenticates) • ExpenseController (authenticates)	Security model. PIN stored as hash, never plain text. Source: US#12

◆ Service / Logic Package

DailyLimitCalculator	• Calculates Safe Daily Limit: remainingBalance / remainingDays • Calculates remaining balance: totalAllowance - totalSpent • Applies daily rollover: redistributes unspent/overspent funds • Checks if spending has crossed the 80% alert threshold • Calculates each category's percentage of total spending • Aggregates all expenses into a Map grouped by Category	• CycleController • ExpenseController • DashboardController • BudgetCycle	Strategy pattern — interchangeable calculation logic. Stateless service. Source: US#3, US#5, FR-2, FR-4
NotificationService	• Tracks whether the 80% alert has already been sent this cycle • Sends local push notification at 80% threshold (sendThresholdAlert)	• ExpenseController (Observer subject) • DashboardController	Observer pattern — reacts to spending events triggered by ExpenseController. Source: US#6, FR-6



Software Design Specification

Class Name	Responsibilities	Collaborators	Pattern / Notes
	- Sends 'Budget Exhausted' critical alert at 100% (sendBudgetExhaustedAlert) - Sends 'Final Day' warning on last day of cycle (sendFinalDayWarning)		

◆ Repository / DB Package

SQLiteHelper	- Maintains single shared database connection (Singleton) - Creates database tables on first launch (onCreate) - Handles database schema migrations (onUpgrade) - Provides writable database connection to repositories - Provides readable database connection for queries	- CycleRepository (depends on) - ExpenseRepository (depends on)	Singleton pattern — only one instance exists. Central DB access point. Source: FR-7
CycleRepository	- Inserts a new BudgetCycle record into SQLite (insert) - Retrieves the currently active cycle (getActiveCycle) - Updates an existing cycle record (update) - Deletes a cycle and cascades to its expenses (delete) - Checks whether any active cycle currently exists (hasActiveCycle)	- SQLiteHelper - BudgetCycle - CycleController	Data access layer for BudgetCycle. Wraps all SQL for cycles. Source: US#1, US#11, FR-7
ExpenseRepository	- Inserts a new Expense record into SQLite (insert) - Updates an existing expense record (update) - Deletes an expense by ID (delete)	- SQLiteHelper - Expense - Category - ExpenseController - DashboardController	Data access layer for Expense. Supports all filtering and aggregation. Source: US#2, US#7-9, FR-3, FR-7



Software Design Specification

Class Name	Responsibilities	Collaborators	Pattern / Notes
	- Retrieves all expenses for a given cycle (getAllByCycle) - Filters expenses by category within a cycle (filterByCategory) - Filters expenses by date range within a cycle (filterByDateRange) - Calculates total amount spent in a cycle (getTotalSpent) - Returns spending totals grouped by Category (getTotalByCategory)		

◆ Controller Package

CycleController	- Validates user input before creating a cycle (validateCycleInput) - Initializes a new BudgetCycle with allowance and dates (initializeCycle) - Retrieves the current active cycle for display (getActiveCycle) - Resets and deletes the current cycle and all its data (resetCycle)	- CycleRepository - DailyLimitCalculator - User	Bridges UI and domain for cycle lifecycle management. Source: US#1, US#11
ExpenseController	- Validates that entered amount is a valid positive number (validateAmount) - Logs a new expense and triggers limit recalculation (logExpense) - Edits an existing expense and updates limit (editExpense) - Deletes an expense and updates limit (deleteExpense) - Retrieves full chronological expense history	- ExpenseRepository - DailyLimitCalculator - NotificationService - User	Observer subject — triggers NotificationService on threshold crossing. Source: US#2, US#7-9



CS251: Phase1-
Project: Masroofy

Software Design Specification

Class Name	Responsibilities	Collaborators	Pattern / Notes
	(getExpenseHistory) • Returns filtered expenses by category or date (getFilteredExpenses)		
DashboardController	• Computes and returns today's Safe Daily Limit (getDailyLimit) • Returns the current remaining balance (getRemainingBalance) • Returns spending breakdown by category as a Map (getCategoryBreakdown) • Checks for a new day and triggers rollover if needed (checkAndTriggerRollover) • Loads all data required to render the dashboard screen (loadDashboardData)	• CycleRepository • ExpenseRepository • DailyLimitCalculator • NotificationService	Strategy user — delegates all calculation to DailyLimitCalculator. Source: US#3, US#4, US#5

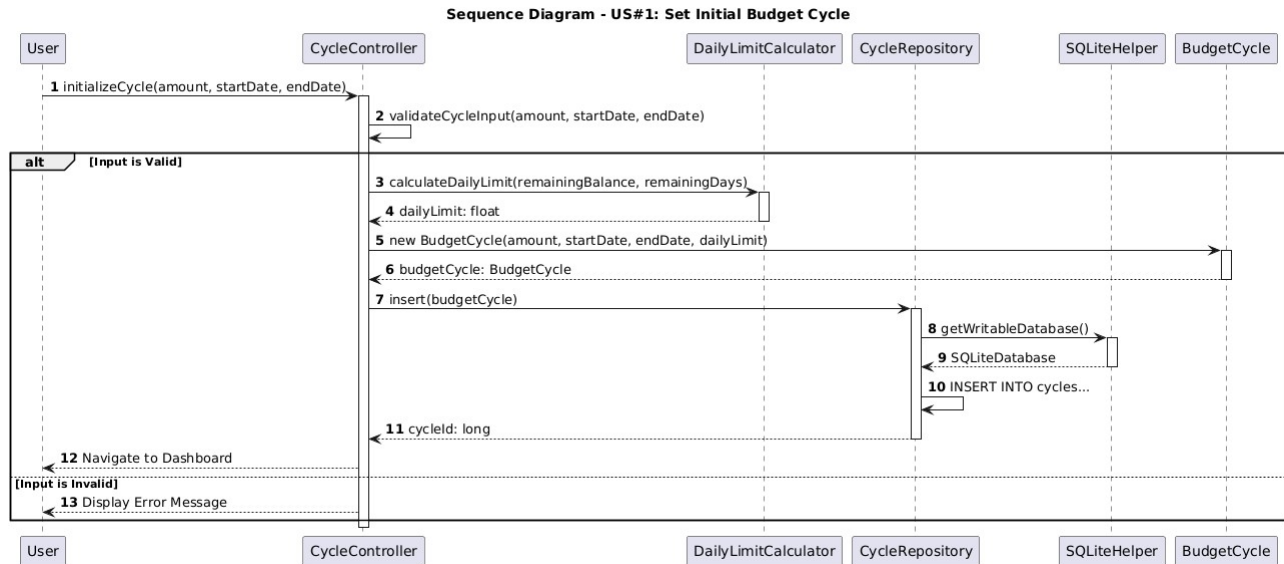


CS251: Phase1- Project: Masroofy

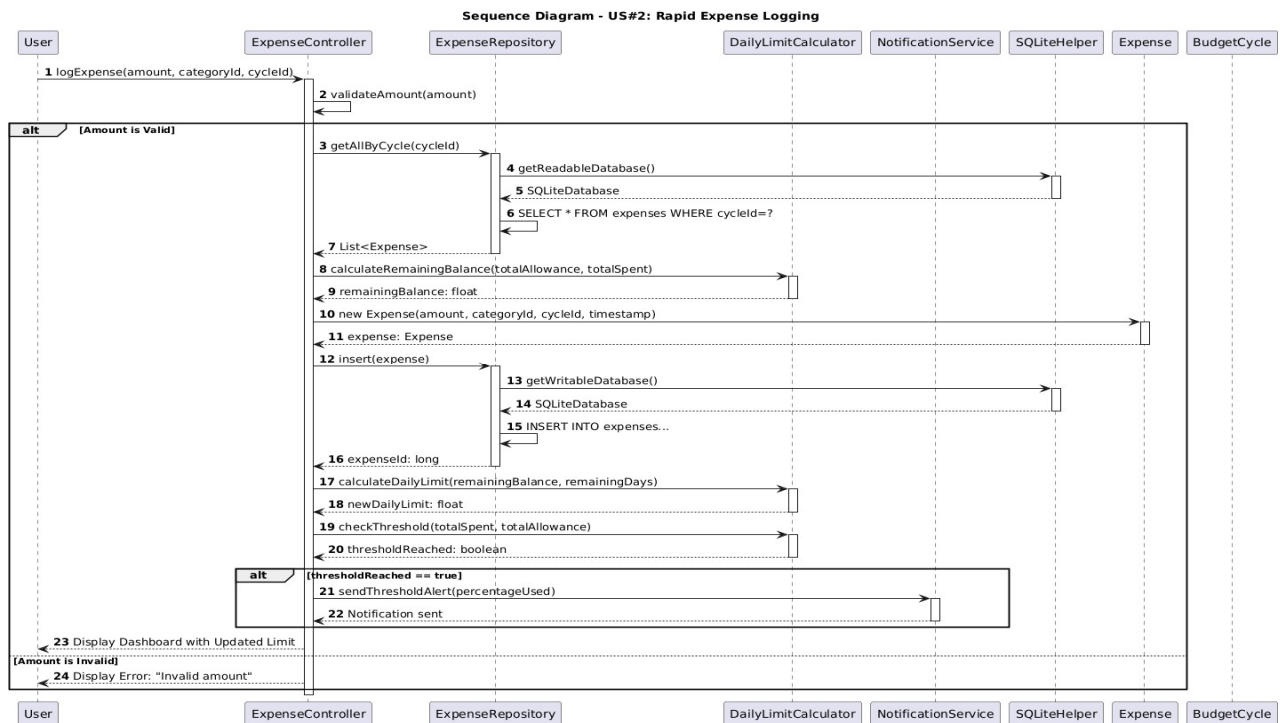
Software Design Specification

IV. Sequence diagrams

USER STORY #1:



USER STORY #2:

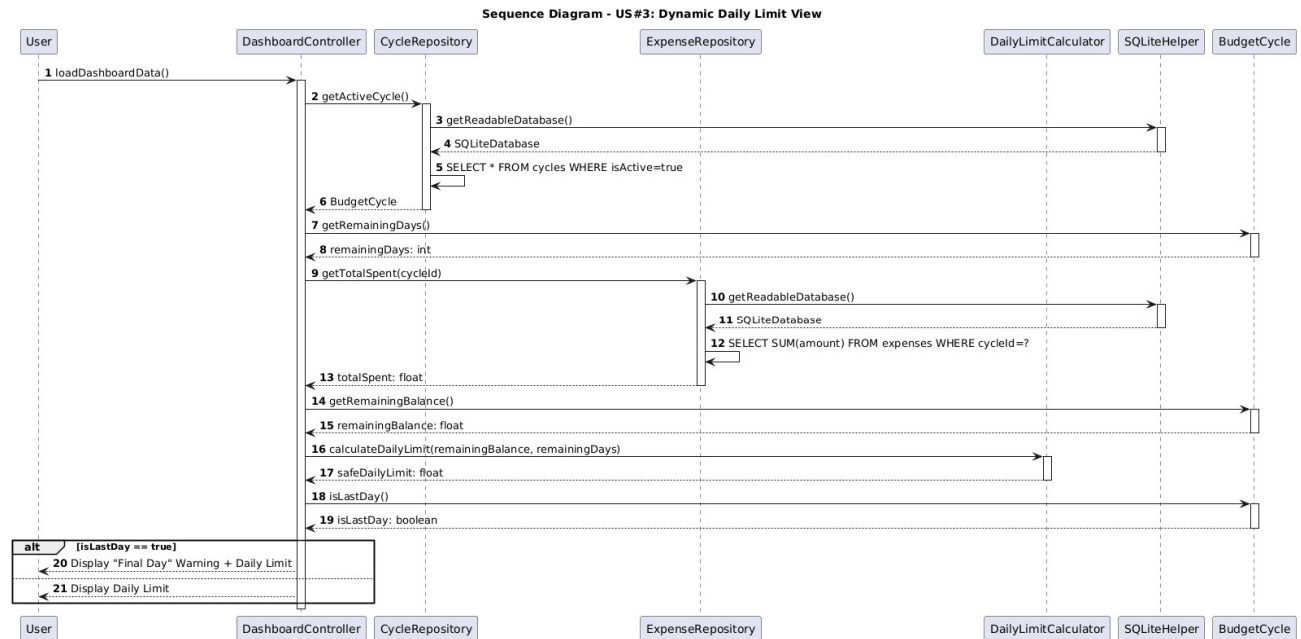


USER STORY #3:

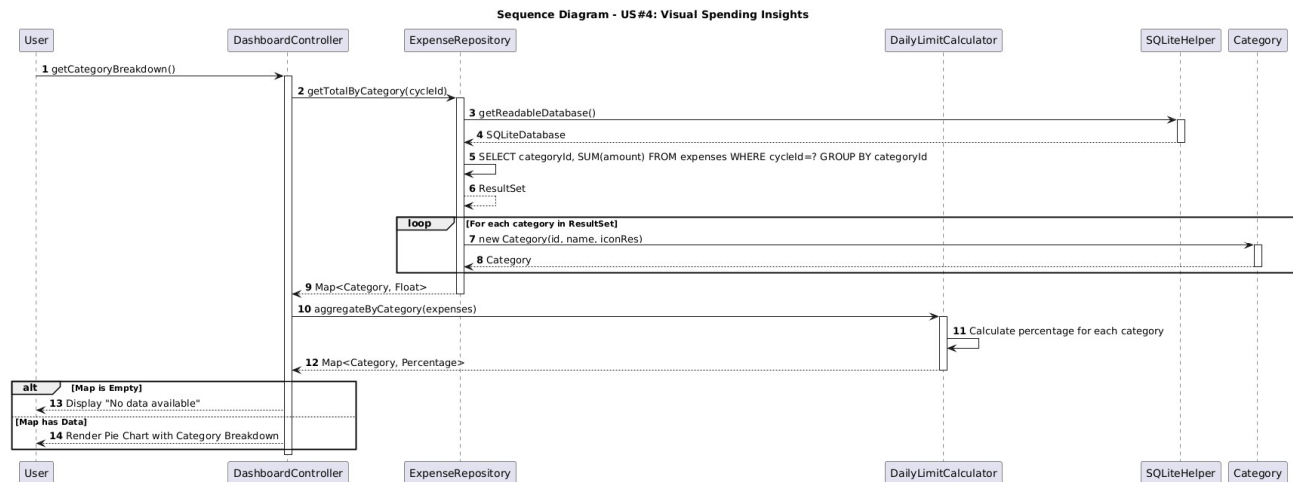


CS251: Phase1- Project: Masroofy

Software Design Specification



USER STORY #4:

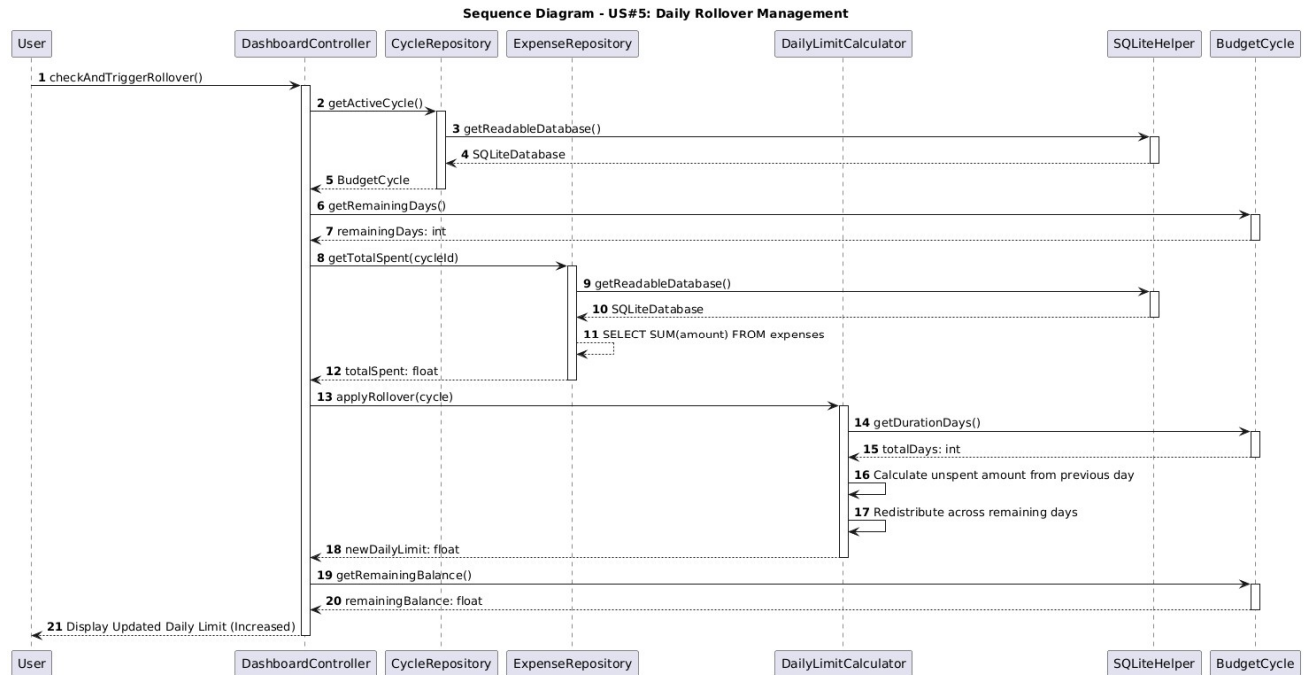




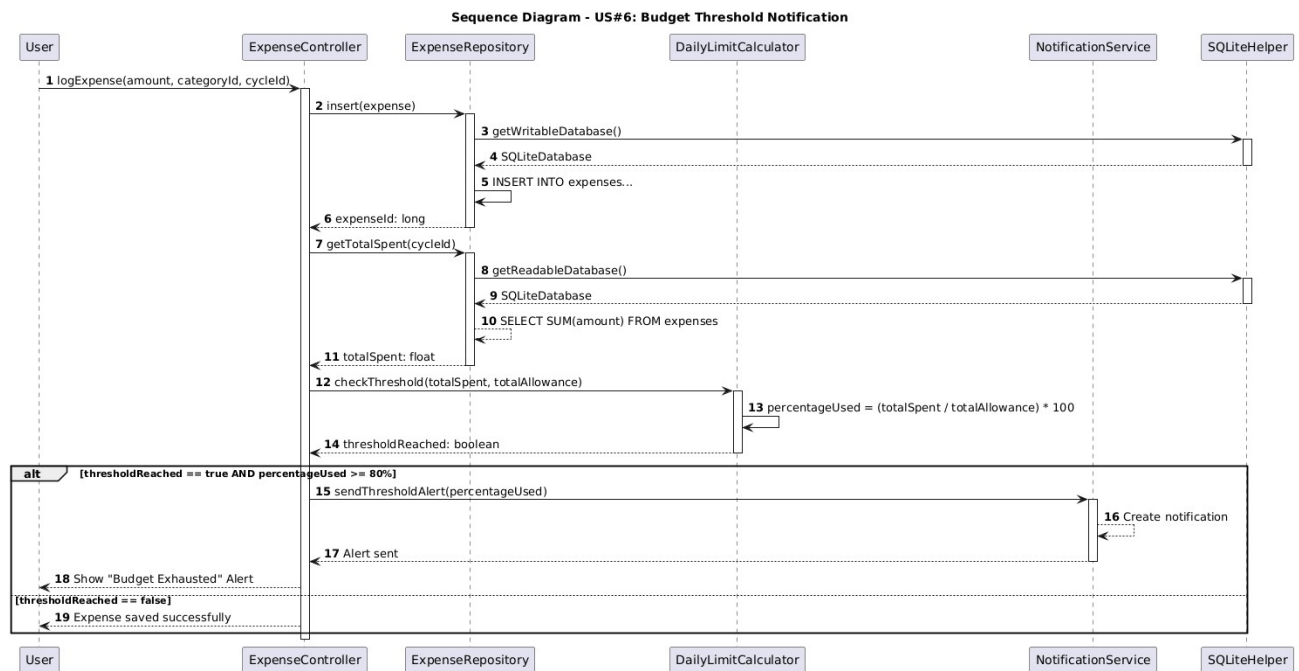
CS251: Phase1- Project: Masroofy

Software Design Specification

USER STORY #5:



USER STORY #6:

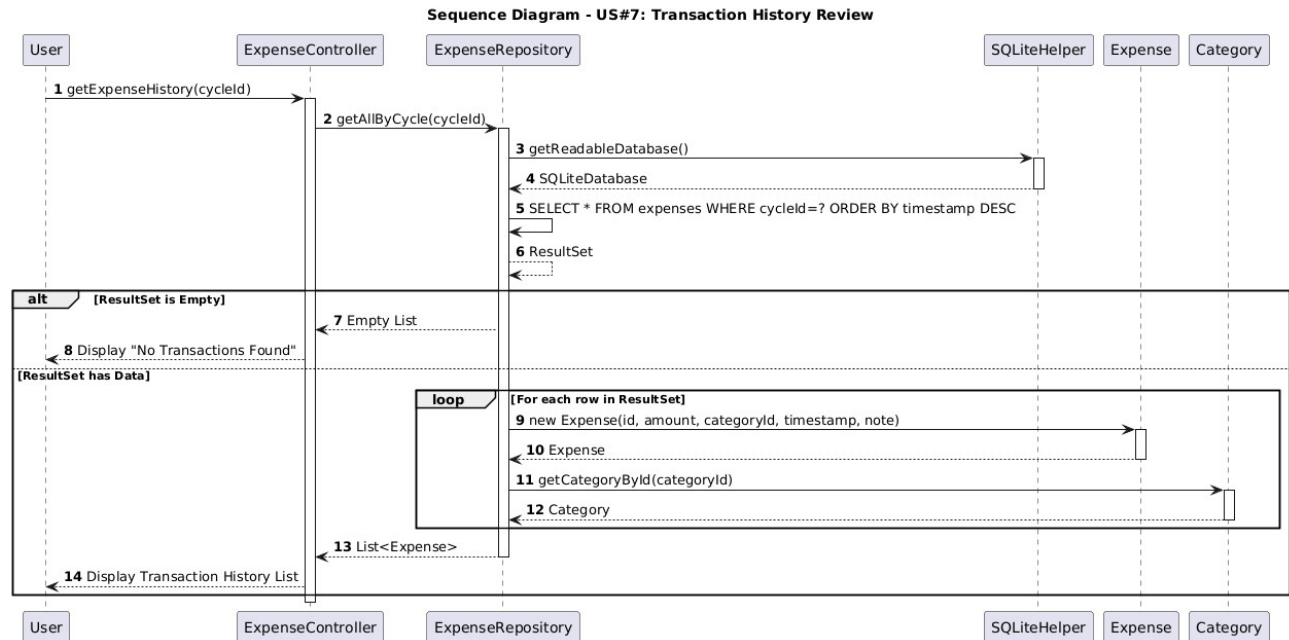




CS251: Phase1-
Project: Masroofy

Software Design Specification

USER STORY #7:



Class – Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
US#1: Set Initial Budget Cycle	CycleController	- initializeCycle(amount, startDate, endDate) - validateCycleInput(amount, startDate, endDate)
	DailyLimitCalculator	calculateDailyLimit(remainingBalance, remainingDays)
	CycleRepository	insert(budgetCycle)
	SQLiteHelper	getWritableDatabase()
	BudgetCycle	new BudgetCycle(amount, startDate, endDate, dailyLimit)



Software Design Specification

Sequence Diagram	Classes Used	All Methods Used
US#2: Rapid Expense Logging	ExpenseController	• logExpense(amount, categoryId, cycleId) • validateAmount(amount)
	ExpenseRepository	• getAllByCycle(cycleId) • insert(expense)
	DailyLimitCalculator	• calculateRemainingBalance(totalAllowance, totalSpent) • calculateDailyLimit(remainingBalance, remainingDays) • checkThreshold(totalSpent, totalAllowance)
	NotificationService	• sendThresholdAlert(percentageUsed)
	SQLiteHelper	• getReadableDatabase() • getWritableDatabase()
	Expense	• new Expense(amount, categoryId, cycleId, timestamp)
	BudgetCycle	• (referenced for totalAllowance)
US#3: Dynamic Daily Limit View	DashboardController	• loadDashboardData()
	CycleRepository	• getActiveCycle()
	ExpenseRepository	• getTotalSpent(cycleId)
	DailyLimitCalculator	• getRemainingBalance() • calculateDailyLimit(remainingBalance,



Software Design Specification

Sequence Diagram	Classes Used	All Methods Used
		remainingDays)
	SQLiteHelper	• getReadableDatabase()
	BudgetCycle	• getRemainingDays() • isLastDay()
US#4: Visual Spending Insights	DashboardController	• getCategoryBreakdown()
	ExpenseRepository	• getTotalByCategory(cycleId)
	DailyLimitCalculator	• aggregateByCategory(expenses) • calculateCategoryPercentage(categoryTotal, totalSpent)
	SQLiteHelper	• getReadableDatabase()
	Category	• new Category(id, name, iconRes)
US#5: Daily Rollover Management	DashboardController	• checkAndTriggerRollover()
	CycleRepository	• getActiveCycle()
	ExpenseRepository	• getTotalSpent(cycleId)
	DailyLimitCalculator	• applyRollover(cycle) • getRemainingBalance()
	SQLiteHelper	• getReadableDatabase()



Software Design Specification

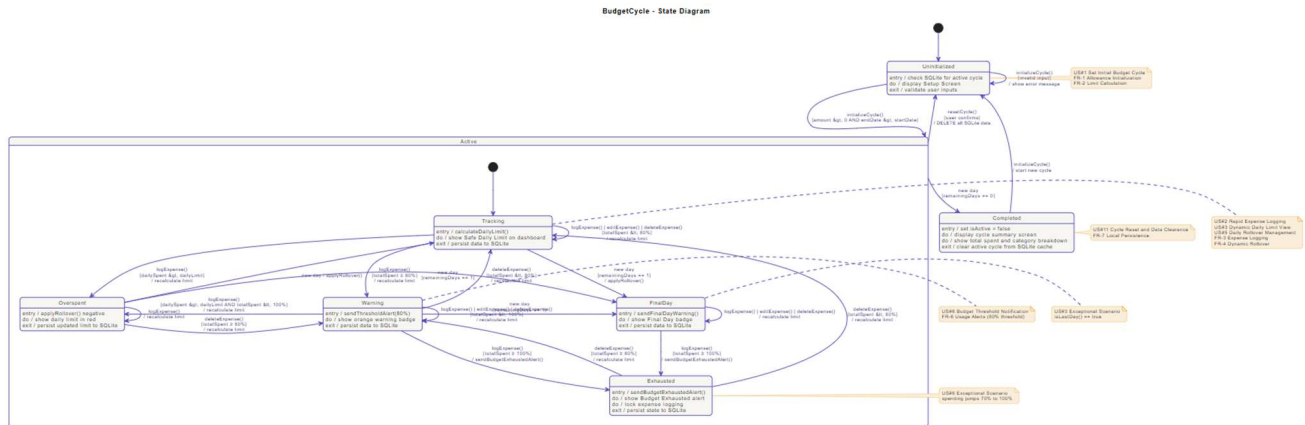
Sequence Diagram	Classes Used	All Methods Used
	BudgetCycle	• getRemainingDays() • getDurationDays()
US#6: Budget Threshold Notification	ExpenseController	• logExpense(amount, categoryId, cycleId)
	ExpenseRepository	• insert(expense) • getTotalSpent(cycleId)
	DailyLimitCalculator	• checkThreshold(totalSpent, totalAllowance)
	NotificationService	• sendThresholdAlert(percentageUsed)
	SQLiteHelper	• getWritableDatabase() • getReadableDatabase()
US#7: Transaction History Review	ExpenseController	• getExpenseHistory(cycleId)
	ExpenseRepository	• getAllByCycle(cycleId)
	SQLiteHelper	• getReadableDatabase()
	Expense	• new Expense(id, amount, categoryId, timestamp, note) • getFormattedDate() • getFormattedAmount()
	Category	• getCategoryById(categoryId)



CS251: Phase1- Project: Masroofy

Software Design Specification

V. State Diagram



VI. Solid Principles

1. Single Responsibility Principle (SRP) Each class in the design has exactly one reason to change. For example, ExpenseRepository is only responsible for database operations on expenses — it does not calculate limits or send notifications. Similarly, DailyLimitCalculator only handles financial calculations and knows nothing about the database or UI. If the calculation logic changes, only DailyLimitCalculator changes. If the DB schema changes, only the repository changes.

2. Open/Closed Principle (OCP) DailyLimitCalculator is used as a Strategy by ExpenseController, DashboardController, and CycleController. If we need a new calculation strategy (e.g., weekly limit instead of daily), we can create a new class that extends or replaces DailyLimitCalculator without modifying the controllers that use it. The controllers are closed for modification but open for extension through the Strategy pattern.

3. Dependency Inversion Principle (DIP) High-level classes like DashboardController and ExpenseController do not depend on low-level database details directly. They depend on ExpenseRepository and CycleRepository as abstraction layers, which internally use SQLiteHelper. The controllers never call SQLiteHelper directly — they only interact with the repository layer. This means if we swap SQLite for a different database, only the repositories change, not the controllers.

VII. Design Patterns

1. Singleton — SQLiteHelper SQLiteHelper is implemented as a Singleton, meaning only one instance of the database connection exists throughout the app's lifecycle. The benefit here is critical for a mobile app: opening multiple database connections simultaneously in Android causes crashes and data corruption. By guaranteeing a single shared instance via getInstance(), all repositories (CycleRepository, ExpenseRepository) safely share the same connection without conflicting.



Software Design Specification

2. Strategy — DailyLimitCalculator DailyLimitCalculator is used as a Strategy by CycleController, ExpenseController, and DashboardController (shown as <<Strategy>> in the class diagram). Instead of each controller implementing its own calculation logic, they all delegate to DailyLimitCalculator. The benefit is that the calculation logic is centralized and interchangeable — if the budgeting algorithm changes (e.g., switching from daily rollover to weekly), only DailyLimitCalculator needs to change, with zero impact on the controllers.

3. Observer — NotificationService & ExpenseController ExpenseController acts as the subject and NotificationService acts as the observer (shown as <<Observer>> in the class diagram). Every time logExpense() is called, ExpenseController checks the spending threshold via DailyLimitCalculator and notifies NotificationService if the 80% or 100% threshold is crossed. The benefit is decoupling: ExpenseController does not need to know how notifications work — it simply triggers the event and NotificationService reacts independently. Adding a new type of alert (e.g., email notification) would only require adding another observer, not changing ExpenseController.

Tools

PlantUML — class diagram, sequence diagrams, architecture diagram, state diagram

Ownership Report

Owners	Items
Eyad Hatem	Sequence Diagrams (1,2,3) ,Class–Sequence Usage Table, Architecture Diagram
Karim Ashraf	Class Diagram, Class Descriptions, Architecture Diagram
Ahmed Sherif	SOLID Principles, State Diagram, Design pattern
Sherif Ahmed	Sequence Diagrams (4,5,6,7), Document purpose and audience, Tools



Software Design Specification

Bonus Points:

Overview

Beyond the required first seven user stories and the core design deliverables, the team implemented additional functionality and a complete production-quality web interface that significantly exceeds the assignment scope. The following bonus work was completed and is fully functional in the submitted implementation.

Bonus 1: Additional User Stories Implementation

The following user stories from the SRS were implemented beyond the required:

US#8 — Edit & Delete Transaction:

A full edit flow was implemented including a dedicated edit page (/expenses/edit/[id]) that pre-fills existing expense data, allows the user to modify amount, category, and note, and calls PUT /api/expense to persist the changes. The delete functionality is also fully connected to the UI via DELETE /api/expense, with the history list refreshing automatically after each deletion.

US#11 — Cycle Reset and Data Clearance:

The DELETE /api/cycle endpoint finds and deletes the active cycle. Because the database schema defines ON DELETE CASCADE on the Expense table's foreign key to BudgetCycle, all related expenses are automatically removed when the cycle is deleted. The Settings page UI exposes this as a "Reset Cycle" action with a confirmation step, matching the exceptional scenario in US#11.

US#12 — Local Privacy Lock (PIN System):

A complete PIN-based authentication system was implemented end-to-end:

- First launch detects whether a PIN exists and routes to sign up or login accordingly
- PIN is hashed using SHA-256 via Node.js crypto module — never stored as plain text
- Failed attempt counter is persisted to the database after every attempt
- Lockout is enforced after 3 consecutive failed attempts for 30 seconds
- A dedicated Change PIN endpoint (PUT /api/user/change-pin) is implemented
- The lockout state survives page refreshes since it is stored in SQLite, not memory



Software Design Specification

Bonus 2: Complete Production-Quality Web UI:

The assignment requires a working backend with OOP and data persistence. The team additionally built a complete, fully connected frontend using Next.js with a consistent professional design system. Every screen is functional and connected to real data from the database.

Design System A Custom Tailwind CSS design system was applied consistently across all pages, including custom typography tokens (font-headline-lg, font-body-sm, font-label-caps), a structured color palette (primary, surface, error, outline variants), and Material Symbols icons throughout.

Dashboard Screen

- Displays the Safe Daily Limit prominently in real time
- Shows remaining days and remaining balance
- A budget progress bar changes color based on usage: green below 80%, orange at 80%, red at 100%
- A donut chart built from real category data using CSS conic-gradient visualizes spending breakdown by category with a live legend
- Alert banners appear automatically when 80%, 100%, or final day thresholds are crossed

Setup Screen

- A live projection card dynamically calculates and displays the initial daily limit as the user types the allowance and selects dates — before even submitting the form
- Validates all inputs before calling the API

Add Expense Screen

- Amount, category, note, and date inputs with consistent styling
- Disabled submit button until an active cycle is confirmed
- Displays current cycle date range and remaining days in a helper card

History Screen

- Transactions grouped by date automatically
- Each expense shows category icon, name, note, and amount
- Edit and delete buttons appear on hover for a clean interface



CS251: Phase1- Project: Masroofy

Software Design Specification

- Delete refreshes the list immediately without a full page reload
- Empty state with a helpful message when no expenses exist

Edit Expense Screen

- Pre-fills all existing expense fields on load
- Calls PUT /api/expense on save and returns to history
- Consistent layout and styling with the Add Expense screen

Settings Screen

- Cycle reset with confirmation
- Change PIN functionality

Authentication Screens

- Signup screen for first-time PIN setup with validation (4 digits, numbers only)
- Login screen with live feedback on wrong attempts and remaining attempts count
- Automatic lockout message with countdown feedback

Responsive Layout All screens use a 12-column grid that collapses gracefully on mobile, ensuring the app is usable on any screen size.

Bonus 3: Additional Next.js Class Diagram:

A second class diagram was produced specifically reflecting the actual Next.js implementation architecture, showing the API Route layer, Services/Business Logic layer, Data Access layer, and Models layer with their relationships and method signatures. This complements the primary Android-style MVC class diagram and demonstrates the team's ability to adapt the design to the chosen technology stack.